
Discovering the Hardening Response of Lithium-Ion Pouch Cells Using Deep Learning

Bachelor Thesis by Cédric Grieder

supervised by Dr. Xueyang Li, Dr. Thomas Tancogne-Dejean

Abstract

This thesis explores the application of deep learning, specifically fully connected neural networks (FCNN), to predict the hardening behavior of lithium-ion pouch cells under mechanical loading. Traditional two-stage analytical models often struggle to capture complex deformation responses, particularly in varied indentation scenarios. By integrating a neural network-based hardening law into finite element simulations, this study aims to improve the prediction of yield stress from plastic strain and enhance force-displacement matching.

This thesis introduces various feature selection methods, focusing on the gradient of the loss function. Various model configurations, machine learning hyperparameters, and selection methods are evaluated to optimize performance. The results demonstrate that while deep learning can effectively model and calibrate for a von Mises yield surface and its associated hardening response, there remains significant room for improvement in achieving consistent accuracy and stability.

Contents

1	Introduction	5
2	Methods	5
2.1	Experiment	5
2.1.1	Performed Experiments	5
2.1.2	Data Preparation	6
2.2	Experiment Simulation	6
2.3	Neural Networks	7
2.3.1	Mathematical Definitions	8
2.3.2	Model Definition	9
2.4	Model Selection	10
2.4.1	Optimization on Two-Stage Hardening Law	10
2.4.2	Selection Process	11
2.5	Loss Function	12
2.6	Filter Contact Elements	15
2.7	Neural Network Training	16
2.7.1	Experiment	16
2.7.2	Scaling	16
2.7.3	Model Initialization	17
2.7.4	Model Prediction	17
2.7.5	Feature Selection	17
2.7.6	Backpropagation	20
2.7.7	Training Loop	21
2.8	Validation	22
3	Results	22
3.1	Conducted Tests	22
3.2	Key Performance Metrics	23
3.3	Training	23
3.4	Validation	27
4	Discussion	28
5	Conclusion	29
6	Outlook	30

1 Introduction

The increasing reliance on lithium-ion batteries, particularly in the automotive industry, necessitates the insurance of safety and performance of these energy storage systems. Lithium-ion pouch cells, widely used in electric vehicles, present specific challenges when subjected to mechanical loading. Large deformations of the mechanically loaded cells lead to potential safety hazards such as fire risks. Thus, modeling the mechanical behavior of these cells under mechanical load is crucial to predicting their response and mitigating potential risks.

Previous studies have used isotropic pressure-dependent plasticity models to describe the mechanical behavior of lithium-ion cells. However, these models often rely on simplified hardening laws that may not fully capture the complexity of the cell's response to deformation. The hardening models, typically expressed through two-stage analytical models, may limit the ability to generalize the response under different loading conditions, particularly for complex geometries or diverse indentation scenarios. This thesis aims to overcome these limitations by leveraging deep learning techniques to predict the hardening response of lithium-ion pouch cells more accurately.

The primary objective of this research is to develop a neural network-based hardening law that can be integrated into finite element simulations, enabling more precise prediction of yield stress from plastic strain. By utilizing a fully connected neural network (FCNN), contrary to traditional hardening models and this approach relies on the neural network's ability to learn nonlinear relationships from data. The proposed model is designed to enhance computational efficiency while providing a more accurate representation of the material behavior, especially in complex indentation scenarios.

This thesis outlines the development of using a deep learning framework for calibrating the hardening law based on experimental data from indentation tests. The neural network is trained to predict the deformation resistance as a function of the equivalent plastic strain, with the calibration process relying on a fast simulation framework. The approach is further extended to investigate the performance of the model for various loading conditions and geometries.

In addition to the primary objective of improving force-displacement matching, this work explores the limitations of current modeling approaches, including convergence issues and the oscillating loss curves observed in previous studies. By incorporating machine learning into the calibration process, this research aims to establish a more robust and general framework for simulating the mechanical behavior of lithium-ion pouch cells. Thus, contributing to advancements in material science and battery technology [Tancogne-Dejean and Mohr, 2022].

2 Methods

The following section outlines the experimental and computational methods employed to investigate the mechanical behavior of lithium-ion pouch cells.

2.1 Experiment

This subsection describes the experimental setup, including the materials, indenter configurations, and procedures used to gather data for training the neural network model.

2.1.1 Performed Experiments

A series of indentation experiments are performed on 4Ah lithium-ion pouch cells, as described in Tancogne-Dejean and Mohr [2022]. This is further illustrated by the balance of forces sketch in Figure 1.

The pouch cells used in the experiments are dimensions of 65.2 mm $\times$ 58.6 mm $\times$ 5.07 mm. The experiments include hemispherical indentations with indenter tip radii of 2.5 mm, 5 mm, 10 mm, 15 mm, 25 mm, and 50 mm, and cylindrical indentations with rod diameters of 5 mm, 10 mm, and 15 mm. To facilitate clear identification of the various experiments, we introduce the following nomenclature: H_D stands for hemispherical indentation, and C_D denotes cylindrical indentation with indenter diameter D mm respectively. For example, C_10 refers to a cylindrical

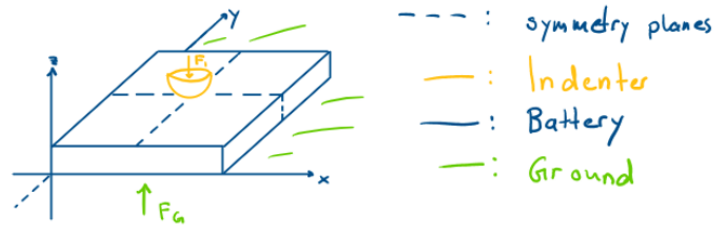


Figure 1: Experiment sketch

indentation using an indenter with a 10 mm diameter. The tests are conducted under quasi-static conditions, which imply no time-dependent kinematic effects.

2.1.2 Data Preparation

The experimental data is processed to focus on the relevant phase of the indentation. Specifically, the data is clipped to start from the initial contact between the indentation tool and the battery cell, defined by the condition where the indenter displacement is equal to zero. Subsequently, the data is truncated at the point of maximum force ($F_{\max}$), where the applied force (F_{exp}) reaches its peak. After partitioning, the time index is normalized so that the data begins at $t_0 = 0$, ensuring consistency across all experiments.

2.2 Experiment Simulation

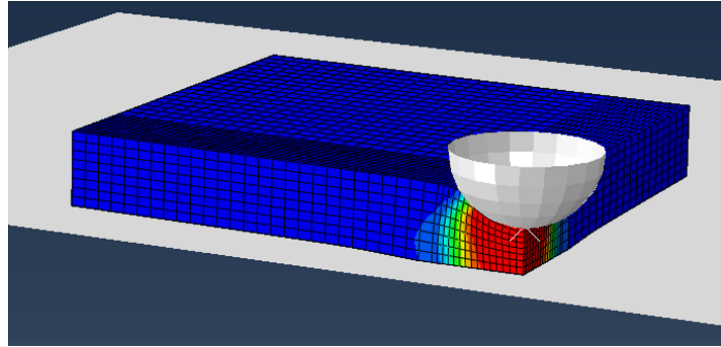


Figure 2: H_10 Simulation

Figure 2 showcases the simulation setup of an H_10 experiment.

Abaqus/Explicit is used to model the battery indentation experiment. To reduce computational time a special mesh is configured. Finite elements, which are presumed to be highly affected by the indenter, have a smaller element size than the rest of the model. Henceforth, this element set will be referred to as “output elements”. Output elements are a size of 0.4mm in both x and y direction and 8 elements in the battery z dimension. The other elements are a size 1.2mm in x, 1mm in y direction and 8 elements for the thickness in z direction.

History output is requested for the output elements and corresponding node set, as illustrated in Figure 3. Extracted variables of the output elements are the stress tensor (σ), plastic strain (ϵ_p), von Mises stress (σ_{vm}). Furthermore, the displacement vector of the output nodes u_{nodes} , z-displacement u_3 and z-reaction-force F_{R3} of the indenter. Default field output is requested for the whole model.

The experiments are conducted under quasi-static conditions therefore mass scaling may be increased, as long as the kinetic energy compared to the total energy has a minor influence ($<5\%$). Mass scaling is applied with a minimal time increment of $d_t := 0.05s$.

The indenter is placed with a 0.1mm spacing to the battery top surface, in the midpoint of the battery’s top xy-surface. The midpoint is chosen as [Tancogne-Dejean and Mohr \[2022\]](#) showed that the geometrical placement of the indenter has a negligible influence on the force response.

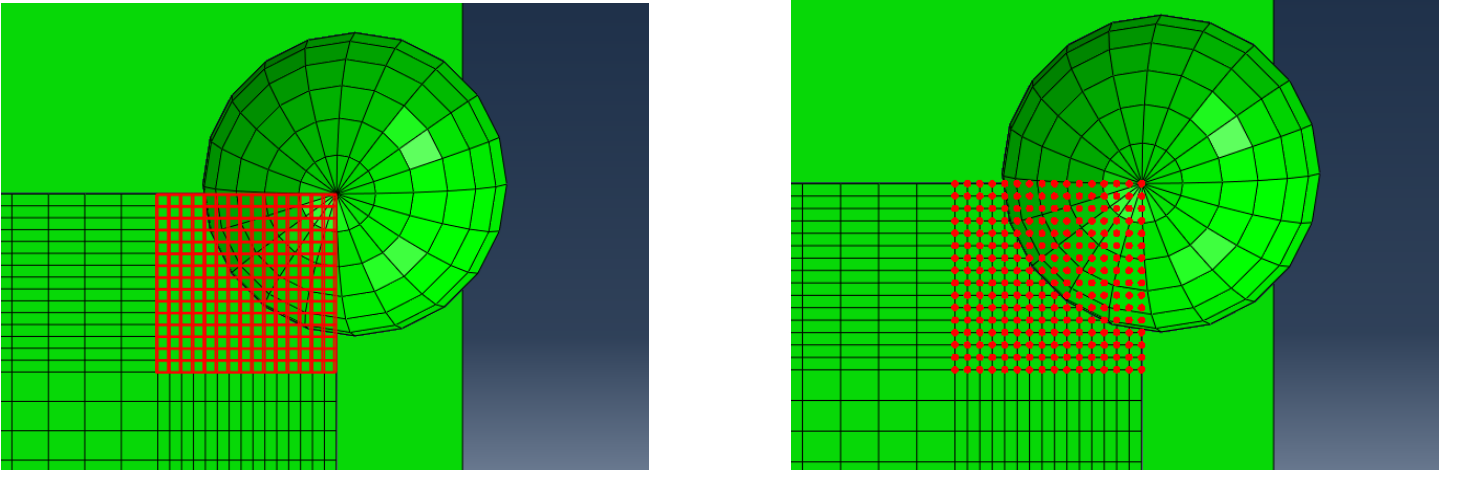


Figure 3: Output element (l) and node set (r)

Experiment type	Indentation depth [mm]
H_10	1.88
H_50	2.01
C_20	1.10

Table 1: Indentation depths of simulated experiments

To further reduce computational time, the symmetry of the battery is exploited and only a quarter of the battery is modeled. Symmetry boundary conditions are applied at the symmetry cut surfaces. The indenter tool and experiment table are modeled as analytical rigid surfaces. Furthermore, the table is fixed in all directions, whereas the indenter is constrained to move in negative z direction only by the specified amplitude. The smooth step amplitude is chosen with varying amplitude maxima depending on the experimental displacement of the respective indentation tool. Table 1 shows the effective indentation displacements (deducted of initial spacing) for performed simulations.

ϵ_p	σ_y [MPa]
0.0	100
0.1	120
0.2	200
0.3	250
0.4	270
0.5	290
0.6	300

Table 2: Copper hardening table

The battery is modeled as a homogeneous solid, using global friction coefficient as proposed by [Tancogne-Dejean and Mohr \[2022\]](#). For the elastic definition of the homogenous battery material Young's modulus $E = 4200$ MPa and Poisson's ratio $\nu = 0.1$ is chosen as previously indicated by [Chung et al. \[2018\]](#). For the initial setup of the simulation a simple copper hardening table, shown in Table 2 is used to define plasticity. This table will be overwritten by the neural network hardening law before every training process. The density (ρ) is assumed to correspond to a homogeneous mixture consisting of equal volumetric proportions (50/50) of copper and aluminum, yielding $\rho = 5.83 \times 10^{-9}$ kg/mm³, has no influence on the results as the simulation is conducted under quasi-static conditions.

2.3 Neural Networks

As a machine learning library, PyTorch is used due to its various built-in functions and user-friendly interface. This makes it a suitable choice for developing and experimenting with deep learning models.

As required by the task, a deep learning framework is developed to predict the hardening law. We use fully connected neural networks (FNNs), also known as multilayer perceptrons (MLPs), as they are straightforward to implement while being capable of handling complex data inputs and modeling nonlinearities effectively.

The hardening law will be modeled by a neural network denoted as k -Neural-Network (kNN). The neural network predicts the yield stress (σ_y) for a given plastic strain (ϵ_p). Both ϵ_p and σ_y are positive scalar values, thus the mapping behavior of the neural network can be expressed as:

$$kNN(\epsilon_p) : \mathbb{R}^+ \rightarrow \mathbb{R}^+, \quad \epsilon_p \mapsto \sigma_y.$$

Neural networks are typically trained using an optimization algorithm known as gradient descent, which minimizes a loss function that quantifies the difference between the predicted and actual values. The process of updating the model's parameters, including its weights and biases, is known as backpropagation. This technique involves propagating the error gradient backward through the network to optimize learning [Bishop, 2006]. Given a set of training data $\{(\epsilon_p^{(i)}, \sigma_y^{(i)})\}_{i=1}^N$, where N is the number of training samples, the loss function ($L(\theta)$) is defined as the mean squared error (MSE) between the predicted yield stress $\hat{\sigma}_y^{(i)}$ and the true yield stress $\sigma_y^{(i)}$:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \left(\hat{\sigma}_y^{(i)} - \sigma_y^{(i)} \right)^2,$$

where θ represents the set of all parameters (weights and biases) in the network.

To minimize the loss, the parameters θ are updated iteratively using the gradient descent algorithm. The update rule for the weights w_{jk} in the network is given by:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial L(\theta)}{\partial w_{ij}},$$

where η is the learning rate, a hyperparameter that controls the step size of each update.

Similarly, the biases b_i are updated as:

$$b_i \leftarrow b_i - \eta \frac{\partial L(\theta)}{\partial b_i}.$$

Backpropagation computes the gradients $\frac{\partial L(\theta)}{\partial w_{ij}}$ and $\frac{\partial L(\theta)}{\partial b_i}$ by applying the chain rule of calculus to propagate the error backward from the output layer to the input layer. This enables the network to adjust its parameters in a way that minimizes the loss function.

The optimization and prediction capabilities of the neural network are therefore highly dependent on the accuracy of the gradient estimates and the effectiveness of the gradient descent scheme. The training process is wrapped in a loop called epochs, where the entire training dataset is passed through the network multiple times, allowing the model to learn and refine its predictions [Bishop, 2006].

2.3.1 Mathematical Definitions

The **Adam** (Adaptive Moment Estimation) optimization algorithm is defined as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{3}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{4}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \tag{5}$$

Where g_t is the gradient at time step t , m_t and v_t are the first and second moment estimates, β_1 and β_2 are the exponential decay rates for these estimates, α is the learning rate, ϵ is a small constant to prevent division by zero, and θ represents the parameters being optimized [Kingma and Ba, 2014].

RMSprop (Root Mean Square Propagation) is defined as:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} g_t.$$

where v_t is the moving average of the squared gradients, β is the decay rate, and the other symbols have the same meanings as for Adam [Hinton et al., 2012].

The **tanh** activation function is defined as:

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

The tanh function maps real-valued numbers into the range $(-1, 1)$ and is commonly used due to its ability to handle negative inputs and outputs symmetrically [PyTorch Documentation, 2023c].

The **softplus** activation function is defined as:

$$\text{softplus}(x) = \ln(1 + e^x)$$

The softplus function is a smooth approximation of the ReLU function, providing a non-linear activation that maps inputs to positive outputs [PyTorch Documentation, 2023b].

Gradient clipping is a technique used to prevent the issue of exploding gradients, which may occur during backpropagation in deep neural networks. When gradients become excessively large, they can cause instability or divergence by making the model's weights update erratically. By ensuring that the gradient norms do not exceed a specified threshold, gradient clipping stabilizes the training process and allows the model to converge more reliably.

This technique is implemented using functions such as `clip_grad_norm`, which rescales the gradients ($\nabla L(\theta)$) if their L_2 norm exceeds a given value, c :

$$\text{if } \|\nabla L(\theta)\|_2 > c, \quad \nabla L(\theta) \leftarrow \nabla L(\theta) \times \frac{c}{\|\nabla L(\theta)\|_2}$$

This ensures that the gradient vector's magnitude remains within the defined limits, effectively preventing the model from making overly large updates to its parameters. The function `clip_grad_norm` thus helps maintain stability during training, particularly in deep or recurrent models, promoting more controlled and stable learning [PyTorch Documentation, 2023a, Zhang et al., 2020].

2.3.2 Model Definition

Here the `SimpleModel` is defined, which corresponds to Model 3 from the chapter 2.4.2 Model Selection. Further the `DeepModel` is defined here which is similar to `SimpleModel` incorporating two more hidden layers for increased model complexity.

SimpleModel The layers in this model are structured as follows:

- **Input Layer:** The input layer (fc1) consists of a linear transformation from 1 to 8 neurons.
- **Hidden Layers:** The network contains two hidden layers:
 - fc2: 8 neurons to 16 neurons.
 - fc3: 16 neurons to 8 neurons.
- **Output Layer:** The final layer (fc4) reduces the output to a single neuron, followed by a Softplus activation to ensure the output is positive.

DeepModel The DeepModel builds upon the SimpleModel by introducing additional layers, increasing the depth and capacity of the network. The architecture is as follows:

- **Input Layer:** The input layer (fc1) has 8 neurons, the same as in SimpleModel.
- **Hidden Layers:** The network features four hidden layers:
 - fc2: 8 neurons to 16 neurons.
 - fc3: 16 neurons to 32 neurons.
 - fc4: 32 neurons to 16 neurons.
 - fc5: 16 neurons to 8 neurons.
- **Output Layer:** The final layer (fc6) outputs a single neuron, followed by the Softplus activation function, similar to SimpleModel.

Like SimpleModel, DeepModel employs Xavier initialization with Tanh activation gain and zero biases. The deeper architecture is intended to capture more complex relationships within the data, making DeepModel suitable for tasks requiring higher model capacity.

In the chapter 2.4.2 Model Selection, a description is given why this model structure was chosen. The SimpleModel is referenced as Model 3.

2.4 Model Selection

2.4.1 Optimization on Two-Stage Hardening Law

To ensure adequate complexity of the chosen machine learning model structure, in this first model selection phase the training capability of different model structures, activation functions and deepness of the model are tested. To test the fitting capability the two-stage-hardening law used previously by [Tancogne-Dejean and Mohr \[2022\]](#) shall be learned by the model. Expressing the Loss as the mean square error of σ_y and $\sigma_{two-stage}$, the predicted yield stress and the two stage hardening law:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \left(\sigma_y^{(i)} - \sigma_{two-stage}^{(i)} \right)^2.$$

The two stage hardening law from [\[Tancogne-Dejean and Mohr, 2022\]](#) is defined as:

$$k_{two-stage}(\epsilon_p) = k_0 + \Delta k$$

$$\Delta k(\epsilon_p) = \begin{cases} A(\epsilon_p)^n & \text{if } \epsilon_p \leq \epsilon_c \\ A\epsilon_c^n + B(\epsilon_p - \epsilon_c) + Q(1 - \exp(-\beta(\epsilon_p - \epsilon_c))) & \text{if } \epsilon_p > \epsilon_c, \end{cases}$$

with parameters

$$\beta = \frac{A n \epsilon_c^{n-1} - B}{Q}$$

$$k_0 = 0.1 \text{ MPa}, \quad A = 1436 \text{ MPa}, \quad n = 1.606$$

$$B = 6.3 \text{ MPa}, \quad Q = 1.0 \text{ MPa}.$$

2.4.2 Selection Process

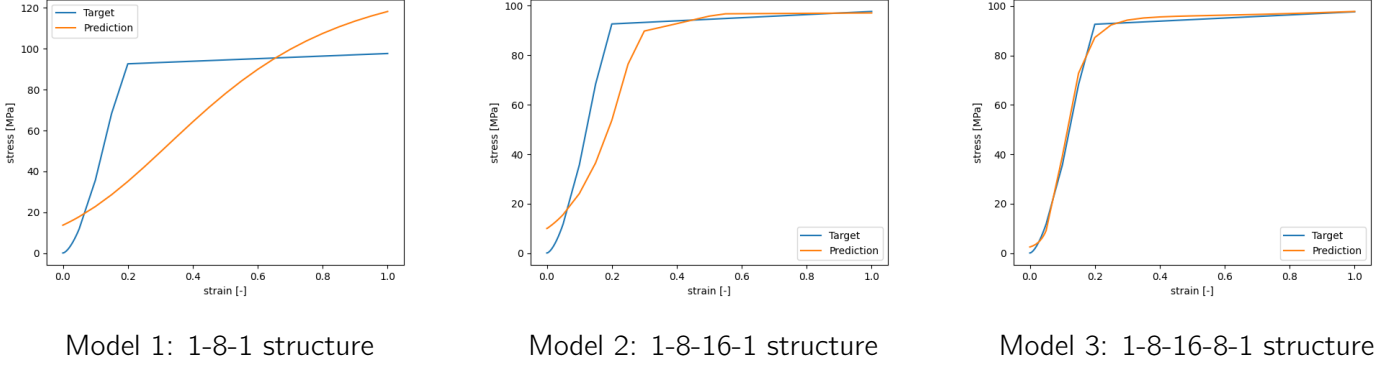


Figure 4: Performance of models with increasing complexity, Blue: Two-stage hardening law. Orange: model yield stress prediction

Systematic exploration of various model configurations by incrementally increasing the number of layers and experimenting with different activation functions is performed.

The architectures tested consist of: Model 1 (1-8-1-softplus), Model 2 (1-8-16-1-softplus), and Model 3 (1-8-16-8-1-softplus). For each model, the performance using the activation functions $\tanh$, ReLU, Sigmoid, and GELU is evaluated. Each configuration is trained over 1000 epochs utilizing a variety of optimization algorithms, including Adam, Adadelta, Adagrad, RMSprop, SGD, AdamW, and NAdam, with a consistent learning rate of 0.001.

After extensive experimentation, Model 3 with the $\tanh$ activation function, using Adam and RMSprop optimization algorithms, provided a "good fit" to the data, demonstrating superior performance across the tested configurations. To illustrate the progression of model complexity, the results of the 3 models with $\tanh$ using the RMSprop optimizer are showcased in Figure 4.

A bottleneck structure refers to a architecture in neural networks where a layer (or a set of layers) has fewer neurons than the preceding and succeeding layers. This constrains the amount of information that can pass through, forcing the network to learn more efficient and abstract representations of the input data.

This selected model structure aligns with established findings on the role of information flow in feedforward neural networks, highlighting the abstraction capabilities of a fully connected neural network using a bottleneck structure [Khadivi et al., 2016].

2.5 Loss Function

The overall goal is to closely approximate the experimental force (F_{exp}) with the simulated force (F_{sim}), therefore a minimization of the difference between these quantities should be achieved.

The loss function is defined as the mean squared error of simulated force and the true experimental force over all indenter displacements (U).

$$L(\theta) = \frac{1}{U} \sum_{u=0}^{u_{\max}} (F_{\text{sim},u} - F_{\text{exp},u})^2.$$

Recalling the definition of the hardening law: $kNN(\epsilon_p) \rightarrow \sigma_y$, it becomes evident that in order to perform backpropagation we need to relate F_{sim} to kNN .

The weight update rule reads:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial L(\theta)}{\partial w_{ij}}.$$

Applying the chain rule yields following expression for the gradient of the loss with respect to the parameter weights:

$$\frac{\partial L(\theta)}{\partial w_{ij}} = \frac{\partial L(\theta)}{\partial F_{\text{sim}}} \frac{\partial F_{\text{sim}}}{\partial w_{ij}}.$$

Differentiating the loss with respect to F_{sim} at a specific displacement:

$$\frac{\partial L(\theta)}{\partial F_{\text{sim}}} = 2(F_{\text{sim}} - F_{\text{exp}}).$$

The partial derivative $\frac{\partial F_{\text{sim}}}{\partial w_{ij}}$ cannot be obtained analytically, as F_{sim} is a nonlinear function derived from the input of the hardening law using the Abaqus simulation. The gradient $\frac{\partial F_{\text{sim}}}{\partial w_{ij}}$ is expressed in terms of $\frac{\partial \sigma_y}{\partial w_{ij}}$, which represents the gradient of the model output and can be directly calculated using backpropagation.

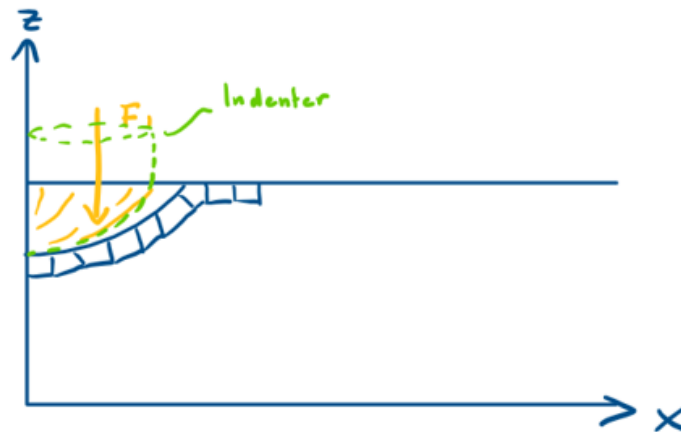


Figure 5: 2D sketch xz-plane

Figure 5 shows a sketch of the surface distributed force (orange) of the indenter exerted on the battery model and deformed finite elements (blue) at a specific displacement.

The static balance of forces of the total indenter force and battery reaction force in z-direction reads:

$$0 = F_{\text{indenter}} + F_{\text{battery}}$$

The battery reaction force is the simulated force:

$$\begin{aligned} F_{\text{battery}} &= F_{\text{sim}} \\ F_{\text{sim}} &= -F_{\text{indenter}}. \end{aligned}$$

Writing the battery reaction force as a sum of n elements forces leads to

$$F_{\text{sim}} = \sum_{k=0}^{n_{\text{elements}}} F_{z,\text{element}, k}.$$

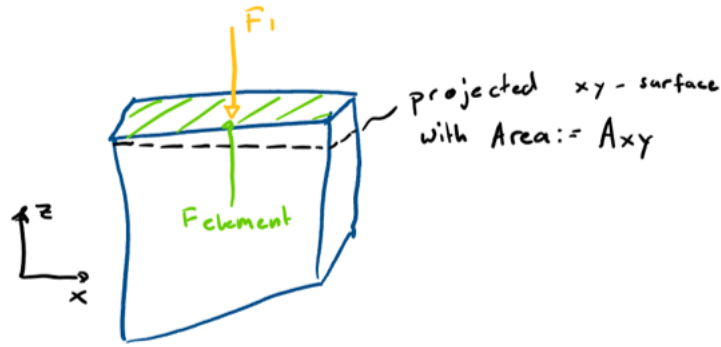


Figure 6: Deformed element sketch

Figure 6 shows a sketch of a single deformed element with element force (green) and the projected xy-area of this element.

Expressing the element force in z direction as the product of stress in force direction and the normal area yields:

$$\begin{aligned} F_{z,\text{element}} &= \sigma_{33} A_{xy}, \\ F_{\text{sim}} &= \sum_{k=0}^{n_{\text{elements}}} (\sigma_{33} A_{xy})_k. \end{aligned}$$

Applying the chain rule for $\frac{\partial F_{\text{sim}}}{\partial w_{ij}}$, assuming A_{xy} to be constant results in

$$\frac{\partial F_{\text{sim}}}{\partial w_{ij}} = \sum_{k=0}^{n_{\text{elements}}} A_{xy,k} \frac{\partial \sigma_{33,k}}{\partial w_{ij}}.$$

Applying the chain rule leads to an expression $\frac{\partial F_{\text{sim}}}{\partial w_{ij}}$ in terms of $\frac{\partial \sigma_y}{\partial w_{ij}}$:

$$\frac{\partial F_{\text{sim}}}{\partial w_{ij}} = \sum_{k=0}^{n_{\text{elements}}} A_{xy,k} \frac{\partial \sigma_{33,k}}{\partial \sigma_y} \frac{\partial \sigma_y}{\partial w_{ij}}.$$

Writing the gradient for the weight update for a single element at a specific displacement enables us to rewrite as follows:

$$\frac{\partial L(\theta)}{\partial w_{ij}} = \frac{\partial L(\theta)}{\partial F_{\text{sim}}} A_{xy} \frac{\partial \sigma_{33}}{\partial \sigma_y} \frac{\partial \sigma_y}{\partial w_{ij}}.$$

The gradient $\frac{\partial \sigma_{33}}{\partial \sigma_y}$ is dependent on the yield surface. Using the von Mises yield criterion, we write:

$$\sigma_y = \sigma_{\text{VM}}.$$

The von Mises stress σ_{VM} is expressed as:

$$\sigma_{\text{VM}} = \sqrt{\frac{1}{2} [(\sigma_{11} - \sigma_{22})^2 + (\sigma_{22} - \sigma_{33})^2 + (\sigma_{33} - \sigma_{11})^2] + 3(\sigma_{12}^2 + \sigma_{23}^2 + \sigma_{31}^2)}.$$

To differentiate σ_{VM} with respect to σ_{33} , the von Mises stress is first rewritten as a square root function:

$$\sigma_{\text{VM}} = \sqrt{g(\sigma_{11}, \sigma_{22}, \sigma_{33}, \sigma_{12}, \sigma_{23}, \sigma_{31})},$$

where

$$g(\sigma_{11}, \sigma_{22}, \sigma_{33}, \sigma_{12}, \sigma_{23}, \sigma_{31}) = \frac{1}{2} [(\sigma_{11} - \sigma_{22})^2 + (\sigma_{22} - \sigma_{33})^2 + (\sigma_{33} - \sigma_{11})^2] + 3(\sigma_{12}^2 + \sigma_{23}^2 + \sigma_{31}^2).$$

The chain rule is then applied to obtain the derivative:

$$\frac{\partial \sigma_{\text{VM}}}{\partial \sigma_{33}} = \frac{1}{2\sigma_{\text{VM}}} \cdot \frac{\partial g}{\partial \sigma_{33}}.$$

Next, the partial derivative of g with respect to σ_{33} is computed. The terms involving σ_{33} are:

$$\frac{\partial g}{\partial \sigma_{33}} = \frac{1}{2} \cdot 2 [(\sigma_{33} - \sigma_{22}) + (\sigma_{33} - \sigma_{11})] = (\sigma_{33} - \sigma_{22}) + (\sigma_{33} - \sigma_{11}).$$

Simplifying then yields:

$$\frac{\partial g}{\partial \sigma_{33}} = 2\sigma_{33} - (\sigma_{11} + \sigma_{22}).$$

Substituting this expression into the chain rule result gives:

$$\frac{\partial \sigma_{\text{VM}}}{\partial \sigma_{33}} = \frac{2\sigma_{33} - (\sigma_{11} + \sigma_{22})}{\sigma_{\text{VM}}}.$$

Inverting the derivative results in:

$$\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}} = \frac{\sigma_{\text{VM}}}{2\sigma_{33} - (\sigma_{11} + \sigma_{22})} = \frac{\partial \sigma_{33}}{\partial \sigma_y}.$$

This expression represents the inverse of the derivative of the von Mises stress with respect to the stress component σ_{33} .

Finally an expression for the total gradient of the loss function with respect to the model weights is found:

$$\frac{\partial L(\theta)}{\partial w_{ij}} = \sum_{u=0}^{u_{\text{max}}} \frac{\partial L(\theta)}{\partial F_{\text{sim}}} \Big|_u \sum_{k=0}^{n_{\text{elements}}} A_{xy} \frac{\partial \sigma_{33}}{\partial \sigma_y} \frac{\partial \sigma_y}{\partial w_{ij}} \Big|_k.$$

2.6 Filter Contact Elements

The filter functions purpose is to select the finite elements that are in direct contact with the indenter tool to get an accurate description of the loss function gradient, $(\frac{\partial L(\theta)}{\partial w_{ij}})$.

As plastic strain ϵ_p is influenced by both geometry and material behavior, the plastic strain for a hardening law kNN is considered dependent on indenter displacement (u_3) as well as the material properties. Elements that have an impact on the force response must undergo plastic strain. However, the condition $\epsilon_p > 0$ fails to filter the correct elements, as some elements deform due to stresses that are not aligned with the force direction.

Further elements which have a direct impact on the force response are expected to have a negative stress normal to the indenter force: $sign(\sigma_{33}) = -sign(F_{indenter}) = sign(F_{battery})$.

However the condition $\sigma_{33} < 0$ fails to filter the correct elements, as some elements experience a negative stress in force direction due to shear stresses of neighbouring elements.

Using the identified conditions, a filtering function may be found with a dynamic threshold for both plastic strain ϵ_p and stress in force direction σ_{33} .

The contact elements are distinguished according to the following criteria:

1. **Initial Condition and Contact-Based Inclusion:** The set of contact elements C is initially empty at $u_3 = 0$. As the displacement u_3 increases, elements that are not filtered (according to the criteria described below) are added to this set. An element i is included in further analysis if it belongs to the set of contact elements C . Mathematically, this is expressed as:

$$i \in C,$$

where $C \subset E$, and E is the set of all elements.

2. **Strain-Based Filtering:** If an element i is not in contact with the indenter surface (i.e., $i \notin C$), it is filtered out if its strain ϵ_i is below the current strain threshold $\epsilon_{threshold}$. The strain threshold is defined dynamically based on the displacement u_3 as:

$$\epsilon_{threshold}(u_3) = \epsilon_{min} + \frac{u_3}{u_{3,max}} \cdot (\epsilon_{max} - \epsilon_{min}).$$

Hence, the strain-based filtering condition is:

$$i \notin C \quad \text{and} \quad \epsilon_i < \epsilon_{threshold}(u_3).$$

Combined Filtering Logic

The overall filtering function is mathematically described by the set of filtered elements F as follows:

$$F = \{i \in E \mid (i \notin C) \wedge [(\epsilon_i < \epsilon_{threshold}(u_3)) \vee (\sigma_{33,i} > \sigma_{threshold}(u_3))]\}.$$

This indicates that an element i is filtered out (i.e., added to the set F) if not in the contact set C . Entailing that either its strain ϵ_i is below the dynamic strain threshold or its stress component $\sigma_{33,i}$ exceeds the dynamic stress threshold.

Updating Contact Elements Elements that are not filtered out are added to the contact set C as the displacement u_3 increases. Thus, the set C is updated dynamically throughout the simulation.

Elements Not Filtered Out The elements that remain for further analysis are those that satisfy the following condition:

$$i \in C \quad \text{or} \quad [i \notin F \text{ and } (\epsilon_i \geq \epsilon_{threshold}(u_3)) \text{ and } (\sigma_{33,i} \leq \sigma_{threshold}(u_3))].$$

Threshold Definitions

Minimum and maximum thresholds that yield promising results are:

- $\epsilon_{\min} = 0.0008$ is the minimum strain threshold.
- $\epsilon_{\max} = 0.1$ is the maximum strain threshold.
- $\sigma_{\min} = -0.5$ MPa is the minimum stress threshold.
- $\sigma_{\max} = -5.0$ MPa is the maximum stress threshold.

The filter function using the specified threshold values is illustrated in Figure 7.

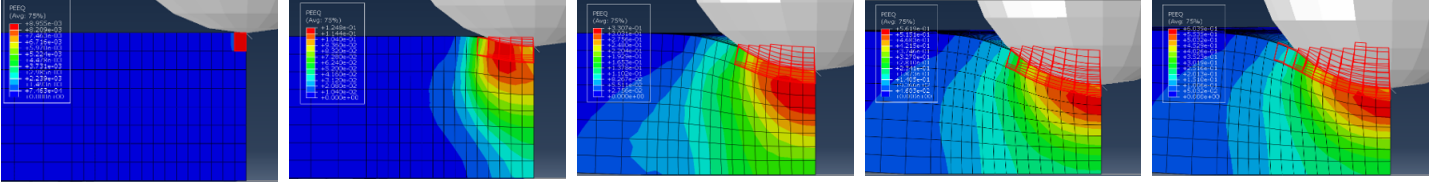


Figure 7: Filter function, contact elements highlighted in red

2.7 Neural Network Training

This section details the training process for the neural network, including the experimental setup, data preprocessing, model initialization, and optimization techniques applied to predict the hardening behavior of lithium-ion pouch cells.

2.7.1 Experiment

The neural network is trained on the H_10 experiment simulation. Opting for a hemispherical indenter with a radius of 5 mm offers a balance between resolution and computational complexity. A smaller indenter would require finer meshing, increasing computational costs, while a larger indenter might reduce the precision of local deformation details. Additionally, this indenter size has been experimentally validated by [Tancogne-Dejean and Mohr \[2022\]](#), providing the data for calibrating the hardening law. This moderate indenter size also ensures that the neural network generalizes effectively across different material behaviors.

2.7.2 Scaling

Input and output scaling is a critical step in neural network training as it ensures optimal model performance during the learning process. By scaling the input and output data to a consistent range, typically between 0 and 1, the network's weights are updated more evenly during backpropagation, reducing the likelihood of gradients vanishing or exploding. This can lead to faster convergence and improved model performance [[Bishop, 2006](#), p. 8]. Scaling the data also helps the neural network to generalize better by preventing the model from becoming biased towards features with larger ranges.

In this particular case, the input scaling is determined by observing the maximum strain value in the Abaqus simulations, which is found to be approximately 0.55. To allow for some margin of error, the maximum value for scaling was set to 0.7. This ensures that even if the maximal strain increases slightly in future simulations, the model will still be able to handle the data without any issues. Similarly, the output scaling is determined by running a simulation based on the two-stage hardening model from [[Tancogne-Dejean and Mohr, 2022](#)]. The maximum observed stress is around 100, so a maximum value of 120 is chosen for scaling, again allowing for some error margin. This careful consideration of the data ranges ensures that the model is robust and accurately predicts outputs even if the input or output values exceed those observed in the initial data set.

2.7.3 Model Initialization

In Glorot and Bengio [2010] the Xavier (Glorot) initialization technique was introduced. Used to address the challenges of training deep neural networks, particularly the issues related to the vanishing and exploding gradients. Traditional random initialization methods often lead to poor convergence due to extreme variations in the layer activations and gradients, causing the network to either learn very slowly or get stuck in suboptimal solutions. The Xavier initialization method aims to maintain the variance of the activations and gradients across layers, ensuring that information flows properly through the network during training. This is achieved by initializing the weights with values drawn from a uniform distribution bounded by $\pm \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$, where n_{in} and n_{out} are the number of input and output units in the layer, respectively. This careful initialization leads to faster convergence and better performance in deep networks.

2.7.4 Model Prediction

As Abaqus requires a table of strain-yield stress values to define the battery material hardening behavior, we use the kNN model to predict the yield stress for a predefined strain tensor. The chosen strain tensor ϵ_p is constructed to cover a wide continuous range of strain values. The strain tensor is defined as follows:

- **Very small strains:** $\epsilon_p \in [0, 0.00095]$ with a uniform spacing of $\Delta\epsilon_p = 0.00005$.
- **Small strains:** $\epsilon_p \in [0.001, 0.0095]$ with a uniform spacing of $\Delta\epsilon_p = 0.0005$.
- **Medium strains:** $\epsilon_p \in [0.01, 0.4]$ with a uniform spacing of $\Delta\epsilon_p = 0.005$.
- **Large strains:** $\epsilon_p \in [0.42, 0.7]$ with a uniform spacing of $\Delta\epsilon_p = 0.02$.

These strain intervals are carefully chosen to ensure a smooth and continuous strain-yield stress relationship. The resulting strain tensor (ϵ_p) is formed by concatenating these intervals to provide a comprehensive set of strain values.

The kNN model, denoted as $\text{kNN}(\epsilon_p) \rightarrow \sigma_y$, is then employed to predict the corresponding yield stress tensor (σ_y). The yield stress tensor, is subsequently used to define the material's hardening behavior in Abaqus.

2.7.5 Feature Selection

Recalling the definition of the loss functions gradient:

$$\frac{\partial L(\theta)}{\partial w_{ij}} = \sum_{u=0}^{u_{\max}} \frac{\partial L(\theta)}{\partial F_{\text{sim}}} \Big|_u \sum_{k=0}^{n_{\text{elements}}} A_{xy} \frac{\partial \sigma_{33}}{\partial \sigma_y} \frac{\partial \sigma_y}{\partial w_{ij}} \Big|_k.$$

The loss functions gradient for a single element at a specific indenter displacement (u) with plastic strain ($\hat{\epsilon}_p$):

$$\frac{\partial L(\theta)}{\partial w_{ij}} \Big|_{\hat{\epsilon}_p} = \frac{\partial L(\theta)}{\partial F_{\text{sim}}} A_{xy} \frac{\partial \sigma_{33}}{\partial \sigma_y} \frac{\partial \sigma_y}{\partial w_{ij}} \Big|_{\hat{\epsilon}_p}.$$

A single neural network input ($\hat{\epsilon}_p$), combined with its corresponding gradient of the loss function $\frac{\partial L(\theta)}{\partial w_{ij}} \Big|_{\hat{\epsilon}_p}$, will be referred to as a **feature**. This is analogous to classical machine learning, where a feature typically consists of an input (often denoted as x_i) and a target or true output (often denoted as y_i).

Recalling the gradient $\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}}$, which is a factor in the total loss function:

$$\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}} = \frac{\sigma_{\text{VM}}}{2\sigma_{33} - (\sigma_{11} + \sigma_{22})}.$$

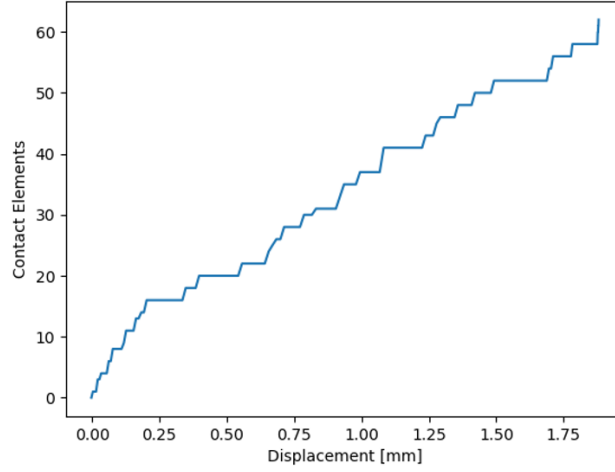


Figure 8: Number of contact elements per displacement

For numerical stability $\tau \ll 1$ is added to the denominator to avoid division by 0:

$$\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}} \approx \frac{\sigma_{\text{VM}}}{2\sigma_{33} - (\sigma_{11} + \sigma_{22}) + \tau}.$$

However for configurations where $(2\sigma_{33} - (\sigma_{11} + \sigma_{22})) \approx 0$ this gradient becomes very large and unstable.

Introducing the concept of **feature selectors**, where the gradient $\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}}$ is adjusted, or features are added or removed to refine the model's performance.

No selector: Referencing "No Selector" means that the features are neither modified nor deleted.

Regularization: Regularization is a concept often used in machine learning where the loss function is modified to include additional terms that penalize certain model characteristics. The primary goal of regularization is to prevent overfitting by discouraging overly complex models that could perform well on training data but poorly on unseen data. This is typically achieved by adding a penalty term to the loss function, such as L1 (Lasso) or L2 (Ridge) regularization, which encourages sparsity in the model coefficients or reduces the magnitude of the coefficients, respectively. By controlling the complexity of the model, regularization helps improve generalization performance, leading to more robust predictions on new data [Bishop, 2006, pp. 256-259].

To balance the number of input features per indenter displacement (u), particularly as with increasing displacements naturally due to the behaviour of the "contact elements filter", a method was implemented where α additional features with $\hat{\epsilon}_p = 0$ as the input were introduced. This was motivated by the observation that the model did not adjust well to certain regions of the force-displacement curve, potentially due to the imbalance in features. See the number of input features per displacement in Figure 8.

Denoting $n(u)$ the number of contact elements dependant on indenter displacement. The function of α was calculated as:

$$\alpha(u) = n(u_{\text{max}}) - n(u).$$

Defining k_0 as $kNN(\hat{\epsilon}_p) = kNN(0)$ and further introducing a scaling factor λ to tune the regularization effect. The gradient of the loss function for regularization features is written as:

$$\left. \frac{\partial L(\theta)}{\partial w_{ij}} \right|_{\hat{\epsilon}_p} = \lambda k_0 \frac{\partial L(\theta)}{\partial F_{\text{sim}}} \frac{\partial k_0}{\partial w_{ij}}.$$

As a result the total loss function writes:

$$\frac{\partial L(\theta)}{\partial w_{ij}} = \sum_{u=0}^{u_{\max}} \frac{\partial L(\theta)}{\partial F_{\text{sim}}} \Big|_u \sum_{k=0}^{n_{\text{elements}}} A_{xy} \frac{\partial \sigma_{33}}{\partial \sigma_y} \frac{\partial \sigma_y}{\partial w_{ij}} \Big|_k + \alpha(u) \lambda k_0 \sum_{u=0}^{u_{\max}} \frac{\partial L(\theta)}{\partial F_{\text{sim}}} \Big|_u \frac{\partial k_0}{\partial w_{ij}}.$$

Setting $\lambda = 0.1$ shows the best model performance in terms of loss compared to choosing $\lambda = 0.05$ and $\lambda = 1$.

This technique acts as a form of regularization, as it constrains the model by reducing the influence of features in regions with fewer active inputs, thereby promoting better generalization across the entire force-displacement curve.

Enforce Direction Another approach is to determine the expected direction of the gradient $\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}}$ by reasoning through a simpler problem.

A simpler case is uniaxial compression, where load is uniformly applied to a sample and stresses act only in the principal direction, with the principal stress σ_3 aligned with the z-axis $\sigma_3 = \sigma_{33}$. For the uniaxial compression case follows:

$$\sigma_1 = \sigma_{11} = \sigma_2 = \sigma_{22} = 0.$$

For uniaxial compression we calculate the gradient with $\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}}$.

The von Mises stress for principal stresses is defined as:

$$\sigma_{\text{VM}} = \sqrt{\frac{1}{2}[(\sigma_1 - \sigma_2)^2 + (\sigma_2 - \sigma_3)^2 + (\sigma_3 - \sigma_1)^2]},$$

with $\sigma_1 = \sigma_2 = 0$ the expression further simplifies to:

$$\sigma_{\text{VM}} = \sqrt{\sigma_3^2} = |\sigma_3|.$$

Since $\sigma_{\text{VM}} = |\sigma_3|$, $\frac{\partial \sigma_3}{\partial \sigma_{\text{VM}}}$ depends on the sign of σ_3 :

- if $\sigma_3 > 0$:

$$\frac{\partial \sigma_3}{\partial \sigma_{\text{VM}}} = \frac{\partial \sigma_3}{\partial \sigma_3} = 1$$

- if $\sigma_3 < 0$:

$$\frac{\partial \sigma_3}{\partial \sigma_{\text{VM}}} = \frac{\partial \sigma_3}{\partial (-\sigma_3)} = -1$$

Thus the gradient $\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}} = \frac{\partial \sigma_3}{\partial \sigma_{\text{VM}}}$ is expressed as:

$$\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}} = \begin{cases} 1, & \text{if } \sigma_{33} > 0 \\ -1, & \text{if } \sigma_{33} < 0 \end{cases}.$$

From this we get the uniaxial compression relationship: $\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}}$ needs to have the same direction as σ_{33} itself.

Let the set S contain all features where $\text{sign}(\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}}) \neq \text{sign}(\sigma_{33})$ holds.

For the feature selector "Enforce Direction", features in S are then modified as:

$$\frac{\partial \sigma_{33}}{\partial \sigma_{\text{VM}}} \Big|_S := \text{sign}(\sigma_{33})$$

2.7.6 Backpropagation

To practically perform backpropagation in PyTorch, features need to be represented as tensors. Opting to use one single tensor for the plastic strain input feature, further referenced as the strain tensor ($\hat{\epsilon}_p$), and correspondingly one tensor for the gradient of the loss function feature, further referenced as the gradient tensor (∇L). The model prediction, the yield stress tensor, is further referenced as the stress tensor. The stress tensor ($\hat{\sigma}_y$) is mathematically given by $kNN(\hat{\epsilon}_p) \rightarrow \hat{\sigma}_y$.

To illustrate how backpropagation is performed in PyTorch using a custom gradient tensor, a minimal code example is given, as this information was surprisingly hard to understand, but very easy to implement. [Community, 2021]

An example strain and gradient tensor are defined, the model is selected and an optimizer is chosen:

```
1 #define the tensors
2 strain_tensor = torch.tensor([0.1, 0.12, 0.2])
3 grad_tensor = torch.tensor([-1, 0, 1])
4
5 #select the model
6 model = SimpleModel()
7 #choose the optimizer, set the learn rate to 0.001
8 optimizer = optim.Adam(model.parameters(), lr=0.001).
```

Further we let our model make a prediction on the strain tensor and capture the resulting stress tensor:

$$kNN(\hat{\epsilon}_p) \rightarrow \hat{\sigma}_y,$$

```
1 #perform prediction
2 stress_tensor = model(strain_tensor).
```

Next, the gradient tensor is propagated backwards through the model on the predicted stress tensor, this is mathematically equivalent to calculating: $\frac{\partial L(\theta)}{\partial \hat{\sigma}_y} \frac{\partial \hat{\sigma}_y}{\partial w_{ij}}$,

```
1 #backpropagation with custom gradient
2 stress_tensor.backward(grad_tensor)
```

Optionally, gradient clipping can be employed before updating the model weights:

```
1 # apply gradient clipping
2 clip = 5
3 torch.nn.utils.clip_grad_norm(model.parameters(), clip)
```

Lastly, we update the model weights using the optimizer: $w_{ij} \leftarrow w_{ij} - \mu \frac{\partial L(\theta)}{\partial w_{ij}}$,

```
1 #updating the model weights
2 optimizer.step()
```

The features used in backpropagation to train the model are dependant on the *feature selector* used. The mechanism however is the same for all features:

- Choose an input feature dependant on indenter displacement u and the element $\hat{\epsilon}_p(u, \text{element})$.
- Use the according *feature selector* algorithm and calculate $\frac{\partial L(\theta)}{\partial \hat{\sigma}_y}(\hat{\epsilon}_p)$.
- Add the single features to the tensors: $\hat{\epsilon}_p \rightarrow \hat{\epsilon}_p, \frac{\partial L(\theta)}{\partial \hat{\sigma}_y}(\hat{\epsilon}_p) \rightarrow \nabla L$.
- Repeat until no features are left.
- Shuffle the tensors $\hat{\epsilon}_p, \nabla L$, which means randomizing their order to prevent possible issues due to a ordered dataset.
- Do a forward pass: $kNN(\hat{\epsilon}_p) \rightarrow \hat{\sigma}_y$.

- Perform backpropagation on the stress tensor with the gradient tensor: $\frac{\partial L(\theta)}{\partial \hat{\sigma}_y} \frac{\partial \hat{\sigma}_y}{\partial w_{ij}}$.
- Apply gradient clipping if specified $\nabla L(\theta) \leftarrow \nabla L(\theta) \times \frac{c}{\|\nabla L(\theta)\|_2}$ (2.3.1).
- Update the model weights using the optimizer: $w_{ij} \leftarrow w_{ij} - \mu \frac{\partial L(\theta)}{\partial w_{ij}}$ (2.3.1).

2.7.7 Training Loop

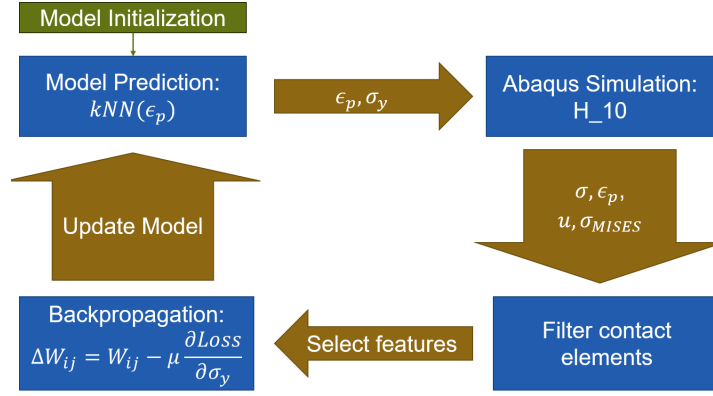


Figure 9: Training loop

In the training loop, which is illustrated in Figure 9, the chosen Neural Network model is trained on the features extracted from the Abaqus simulation. The training loop is run for a fixed number of epochs and a fixed configuration of the training parameters thereof being:

- **Number of epochs:** How many times the training loop is run. When running training on 8 CPU's on the Euler HPC one epoch takes approximately 1 minute. Depending on the *feature selector*, clip value, learn rate, model, optimizer, the lowest loss / best hardening law (*kNN*) may be found earlier or later. Typically epoch number is kept between 100 and 1000, 1600 is the maximum value tested, showing no significant improvement compared to 1000.
- **Learn rate (μ):** different weight update parameter sizes were used, these are the initial learn rates which both RMSprop and Adam optimizer adjust based on the gradients, as described previously. Still the chosen value matters here, experimentation yielded $\mu \in \{10^{-3}, 5 \cdot 10^{-4}\}$ as the best performing values. Whilst higher values lead to unstable training behaviour and lower values give slow convergence for which the number of epochs becomes extensively high.
- **Clip value:** Tested values are: $\{\text{None}, 0.1, 0.5, 1, 2, 5, 10, 15\}$. Generally clip values lower than 5 yield similar results to very low learn rates, decreasing model convergence speed whilst not improving stability noticeably. Using no clip value generally makes the training process unstable and leads to divergent solutions. Clip values $\{5, 10, 15\}$ are primarily used as they balanced convergence and stability for the feature selectors: *Regularization*, *Enforce Direction*

Alternative clipping techniques are tested as well using *No Selector* most notably:

Directly clipping the gradient features (∇L) if the absolute value exceeds a certain threshold. No useful threshold is found. Also scaling down loss gradients of features which has significant outlier values in its gradient ∇L (more than two standard deviations of all features in ∇L) does not improve the training behaviour.

As no specific training parameters are found which yield a monotonically decreasing loss curve and generally convergence is not achieved, meaning that the model starts diverging after reaching a local minima, the training loop is run for a fixed number of epochs. The best model is chosen after training, best meaning loss in terms of mean squared error of experimental and simulation force respectively. Figure 10 is showcasing an example loss curve, illustrating the non-monotonic behaviour.

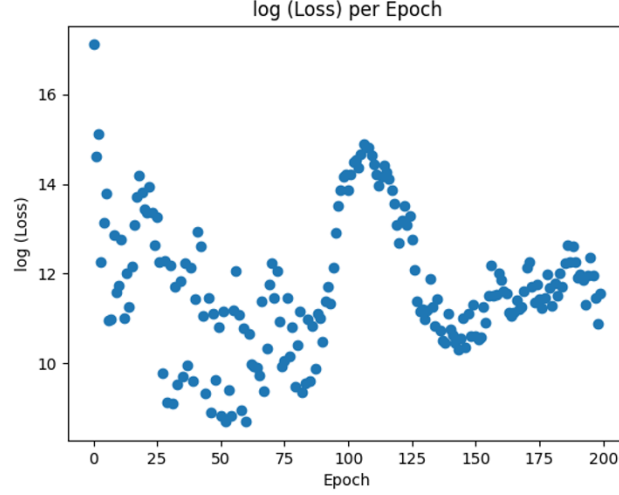


Figure 10: Logarithm of the loss against epoch number, DeepModel, *Regularization*

2.8 Validation

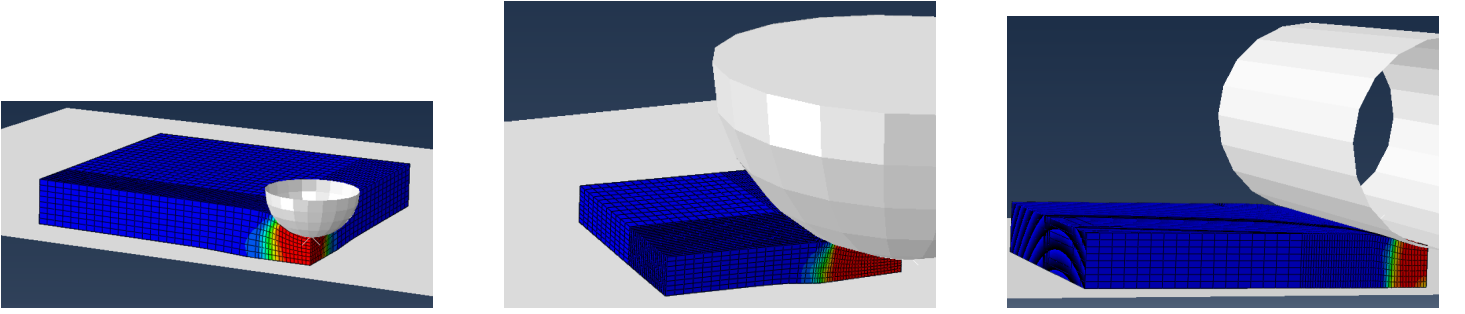


Figure 11: H_10, H_50, C_20 simulation setup (l) to (r)

For the validation of selected neural network hardening laws (kNN), validation runs are performed on H_50 and C_20 indentation experiment simulations. These are chosen as H_50 reflects a significant size difference of the indenter tool and C_20 reflects another shape and size (cylindrical instead of hemispherical). The three different simulation setups are illustrated in Figure 11.

3 Results

This section presents the results obtained from evaluating the performance of the SimpleModel and DeepModel under various configurations, focusing on feature selection strategies, learning rates, optimizers, and gradient clipping techniques. The key performance metrics, including MAE, MAPE, and MSE, are used to assess and compare the models' effectiveness in minimizing the force error between the simulated and experimental results.

3.1 Conducted Tests

This section presents the performance evaluation of SimpleModel and DeepModel across the three previously introduced feature selection strategies. Each configuration is run for 600 epochs to ensure comparability. The learning rate is set at either 0.0005 or 0.001, with RMSprop used for the *Regularization* configuration and Adam for the *No Selector* and *Enforce Direction* configurations. Gradient clipping values of 5, 10, and 15 are used for Regularization, No Selector, and Enforce Direction, respectively. These configurations are selected based on previous experiments to optimize the models' performance and to facilitate a balanced comparison.

Model	Feature Selector	Learn rate (lr/μ)	Optimizer	Clipping Value	Epochs
SimpleModel	No Selector	0.001	Adam	10	600
SimpleModel	Regularization	0.0005 / 0.001	RMSprop	5	600
SimpleModel	Enforce Direction	0.001	Adam	15	600
DeepModel	No Selector	0.001	Adam	10	600
DeepModel	Regularization	0.0005 / 0.001	RMSprop	5	600
DeepModel	Enforce Direction	0.001	Adam	15	600

Table 3: Comparison of SimpleModel and DeepModel across different feature selectors, learning rates, optimizers, and gradient clipping values.

3.2 Key Performance Metrics

For the performance evaluation following metrics are used: The mean absolute error (MAE), the mean percentage error (MAPE) and the mean square error (MSE). Key performance metrics (KPM): Lower Scalar results for all performance metrics mean "better results" in Terms of minimizing the force error between simulated and experimental force.

$$\begin{aligned}
MAE(F_{\text{exp}}, F_{\text{sim}}) &:= \frac{1}{n} \sum_{i=1}^n |F_{\text{exp},i} - F_{\text{sim},i}| \\
MAPE(F_{\text{exp}}, F_{\text{sim}}) &:= \frac{100\%}{n} \sum_{i=1}^n \left| \frac{F_{\text{exp},i} - F_{\text{sim},i}}{F_{\text{exp},i}} \right| \\
MSE(F_{\text{exp}}, F_{\text{sim}}) &:= \frac{1}{n} \sum_{i=1}^n (F_{\text{exp},i} - F_{\text{sim},i})^2.
\end{aligned}$$

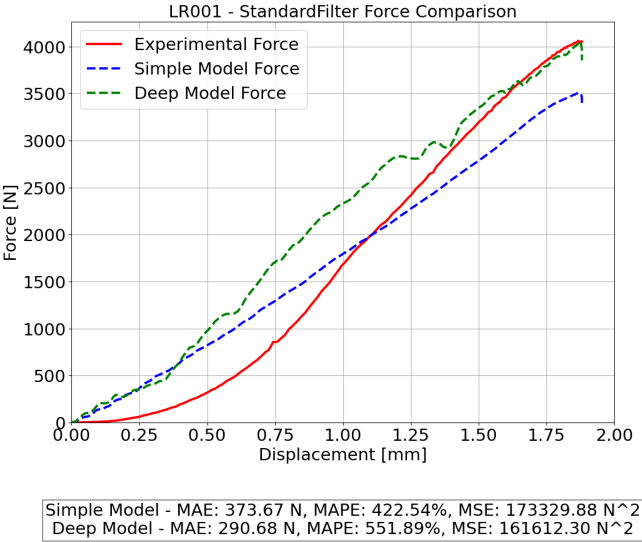
3.3 Training

Table 4: Performance metrics for different models and feature Selectors (rounded to 3 digits)

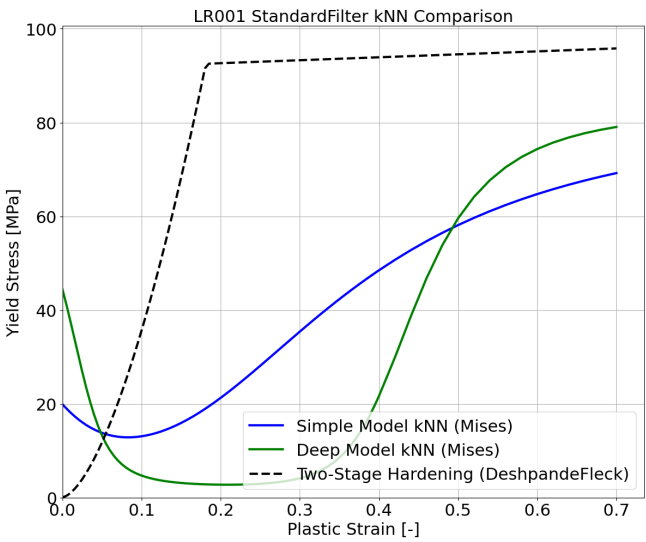
Feature Selector	Model	Learn Rate	Epoch at Lowest Loss	MSE	MAPE	MAE
EnforceS33Direction	DeepModel	LR0005	n.a. ¹	6.31×10^2	74.4	21.21
EnforceS33Direction	DeepModel	LR001	258	1.81×10^3	57.9	34.7
Regularization	DeepModel	LR001	69	1.94×10^3	41.5	36.6
Regularization	DeepModel	LR0005	581	3.66×10^3	52.8	53.0
EnforceS33Direction	SimpleModel	LR001	590	5.70×10^3	63.2	64.4
Regularization	SimpleModel	LR0005	538	1.40×10^4	93.0	102
Regularization	SimpleModel	LR001	304	1.86×10^4	91.1	118
EnforceS33Direction	SimpleModel	LR0005	516	1.90×10^4	85.2	121
StandardFilter	DeepModel	LR001	535	1.62×10^5	293	291
StandardFilter	SimpleModel	LR001	599	1.73×10^5	239	374

¹n.a.: Data point not available for this entry.

Training Results for No Selector



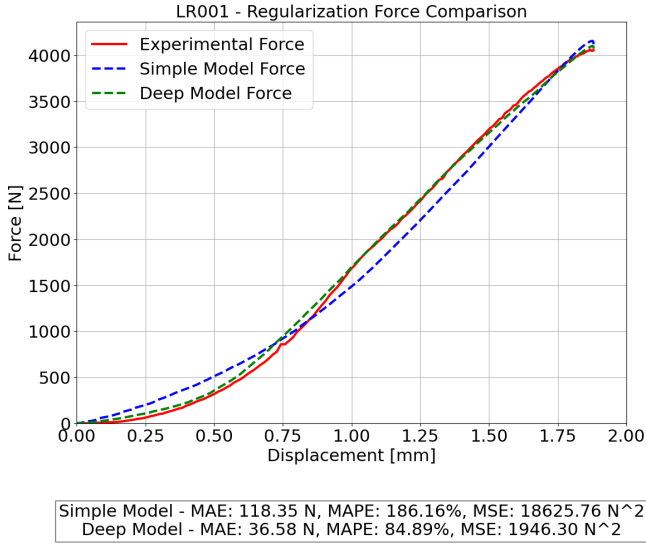
(a) Force Comparison



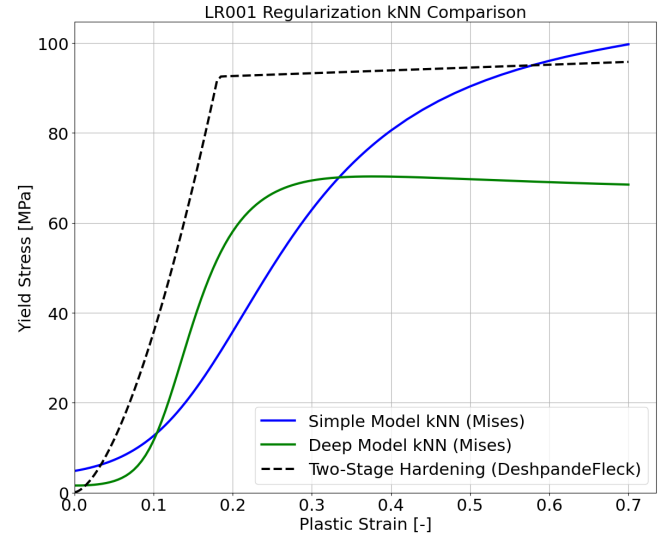
(b) kNN Comparison

Figure 12: Training results without feature selector (No Selector) at LR 0.001.

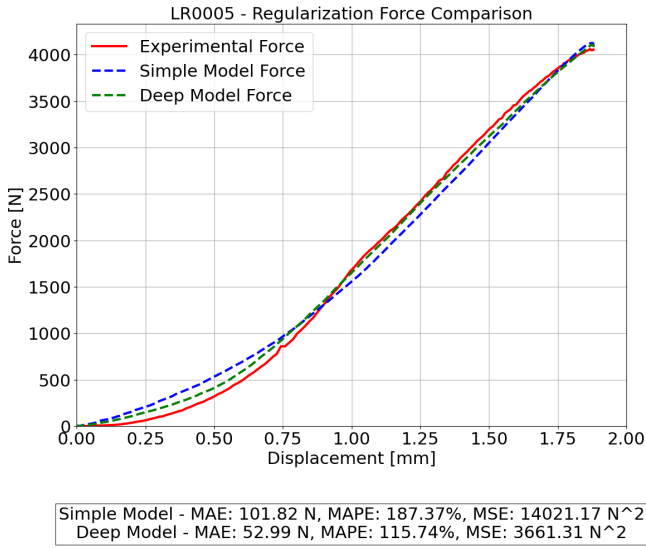
Training Results for Regularization



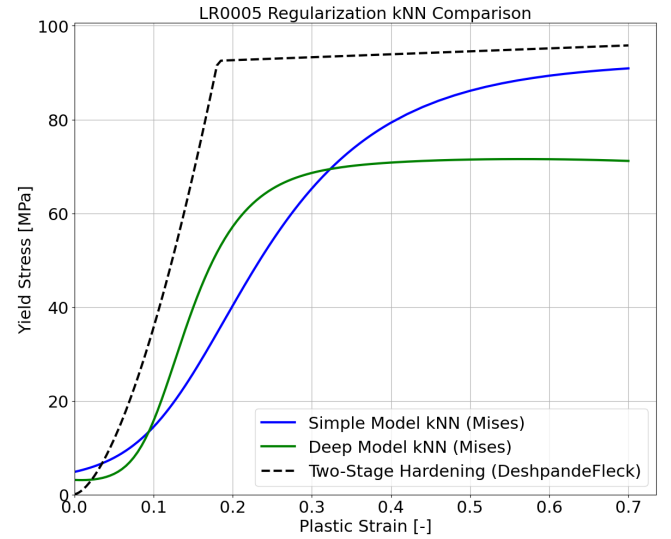
(a) Force Comparison for LR 0.001



(b) kNN Comparison for LR 0.001



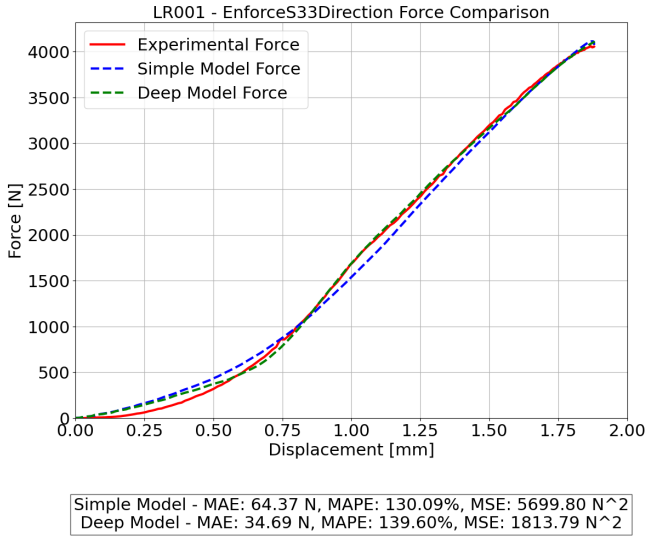
(c) Force Comparison for LR 0.0005



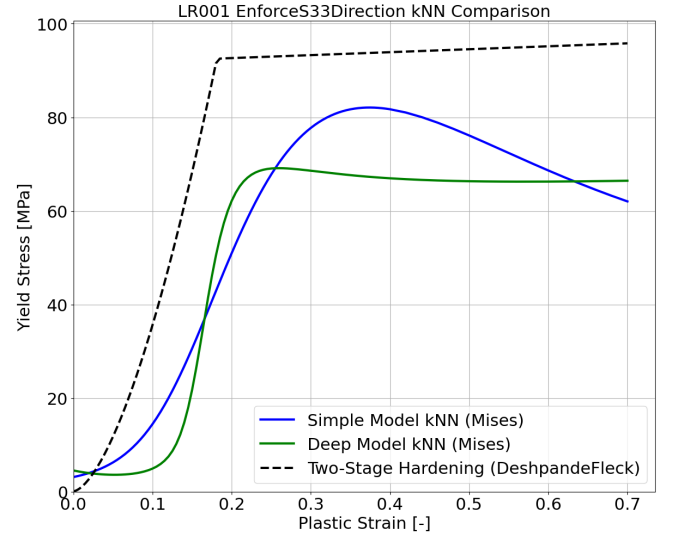
(d) kNN Comparison for LR 0.0005

Figure 13: Training results with regularization at different learning rates (LR 0.001 and LR 0.0005).

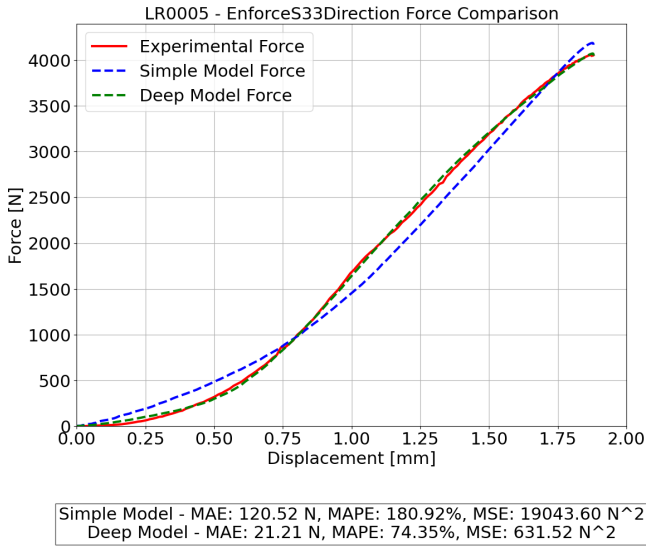
Training Results for Enforce Direction



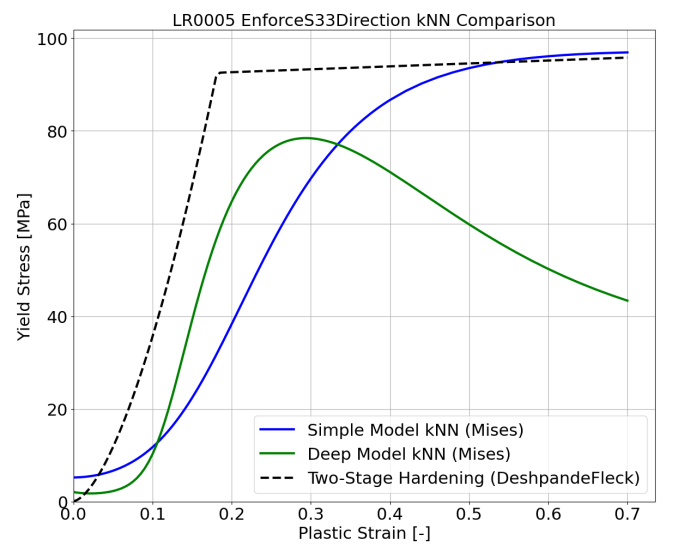
(a) Force Comparison for LR 0.001



(b) kNN Comparison for LR 0.001



(c) Force Comparison for LR 0.0005



(d) kNN Comparison for LR 0.0005

Figure 14: Training results with Enforce Direction at different learning rates (LR 0.001 and LR 0.0005).

The Key Performance Metrics and respective configurations of training runs are indicated in Table 4. Figures 12, 13, 14 show the training results for the specified method combinations. For shape comparison the two stage hardening law is included. The amplitude is not comparable as it was calibrated on a Deshpande-Fleck yield surface. Displayed are the results for the epoch, out of the total 600 epochs, that yield the lowest loss (MSE).

Further the MSE, MAPE and MAE for specified epochs and training combinations is displayed in the table. The best run, based on the lowest performance metrics, was achieved by DeepModel with a learning rate of 0.005 and the Enforce Direction feature selector.

In contrast, the worst performance across all key performance metrics (KPM) was observed with DeepModel using a learning rate of 0.001 and the No Selector method. Overall, DeepModel with a learning rate of 0.005 and the Enforce Direction feature selector consistently produced the best results, minimizing error metrics across MSE and MAE performance measures.

The described simulations took 8 hours, running on Euler HPC, yielding a runtime of approximately one minute per epoch.

DeepModel consistently performs better than SimpleModel, as presented in figures 12, 13, 14 for the training runs on the H_10 indenter except for the No Selector method. DeepModel predicted hardening laws with regularization and enforce direction are chosen to be validated on the H_50 and C_20 indenters. Regularization models are used as they show lower MAPE.

3.4 Validation

Feature Selector	Model	Learn Rate	Scenario	MSE	MAPE	MAE
Enforce Direction	DeepModel	LR001	C_20	3.00×10^7	201	4300
			H_50	4.60×10^5	275	572
Enforce Direction	DeepModel	LR0005	C_20	6.87×10^6	128	2136
			H_50	5.51×10^4	147	153
Regularization	DeepModel	LR001	C_20	1.92×10^7	122	3281
			H_50	2.02×10^6	138	1049
Regularization	DeepModel	LR0005	C_20	2.43×10^7	168	3562
			H_50	6.79×10^5	229	663

Table 5: Performance metrics for DeepModel validation (rounded to 3 digits)

Key performance metrics of validation experiments are presented in Table 5. The lowest MSE for the cylindrical and hemispherical indentation is achieved by the best training configuration (Enforce Direction, DeepModel, LR0005). As for the training runs the MAPE is lower for Regularization than for Enforce Direction. Figure 15 shows validation results for both learn rates across configurations in one chart.

Validation Results

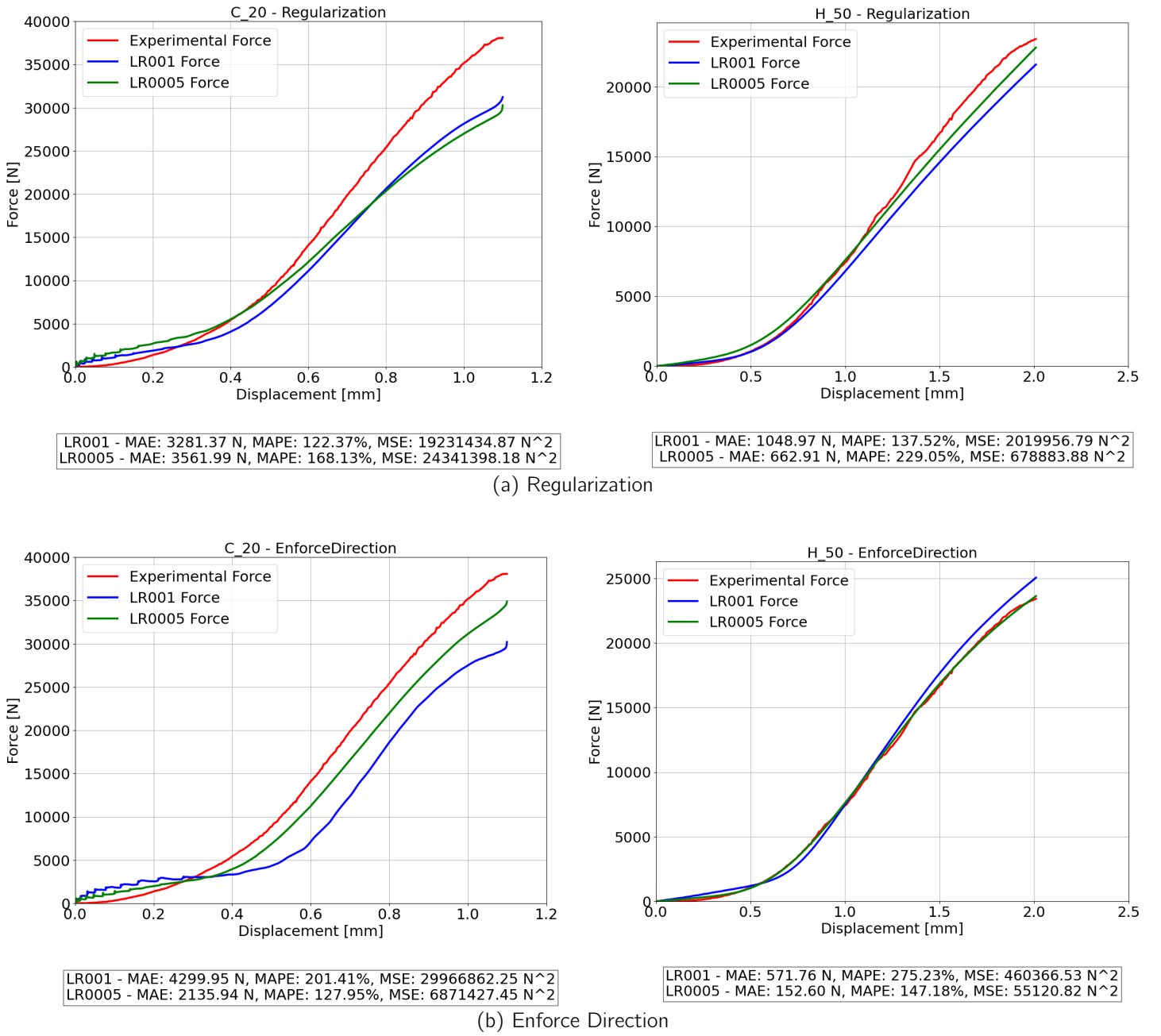


Figure 15: Validation results for selected hardening laws

4 Discussion

The calibration process using *DeepModel* is feasible but highlights limitations when using configurations such as no selector, which yields no successful neural network calibration, particularly for deeper models. This is likely due to overfitting caused by unsmooth data or coarse meshing. The gradient $\frac{\partial \sigma_{33}}{\partial \sigma_{VM}}$ is also likely impacted by the unsmooth data, leading to poor results.

In contrast, the enforce direction feature selector produces better results, achieving more accurate gradient representations when simplifying boundary conditions, such as uniaxial compression. However, it occasionally results in decreasing yield stress at higher strains. regularization shows merit in handling these issues, especially at lower displacements, where it provides a more stable force match.

A key issue is the uneven force response at lower displacements, resulting in a high MAPE across the force-displacement curve. This is likely caused by the inclusion of more contact elements at higher displacements, leading to an uneven gradient distribution. If a smoother and evenly distributed gradient representation for the loss function could be achieved, convergence issues would likely be reduced, leading to stable and reliable training.

The oscillating loss curves are concerning, likely due to large steps in the gradient descent or uneven feature distribution. A method to improve feature selection and machine learning hyperparameters could stabilize the optimization process, leading to a more accurate force response.

5 Conclusion

The main objective of this thesis was to investigate the use of deep learning models, specifically fully connected neural networks (FCNN), to predict the hardening law of lithium-ion batteries. The goal was to optimize force matching in simulations by predicting yield stress from plastic strain.

This research demonstrated that:

- The calibration process using DeepModel was feasible, achieving successful force and displacement matching.
- Enforce direction outperformed other configurations, particularly in terms of accuracy in validation runs such as C_20 and H_50, while regularization showed better performance at lower displacements.
- Challenges with convergence and oscillating loss curves were encountered, particularly in the no selector configuration.

This study contributes to the field of material science and battery technology by applying deep learning techniques to model complex material behavior. The successful application of FCNN models to simulate the hardening behavior of pouch cells presents a new approach to improving computational efficiency in simulation environments like *Abaqus*. Furthermore, the framework developed in this work may be applied to any calibration process if an adequate loss function gradient is identified. Using more advanced yield surfaces beyond the *von Mises criterion* could further enhance efficiency and reduce the need for manual calibration.

Several limitations are identified, including unstable training behavior, lower accuracy at reduced displacements, and issues arising from mesh size and data density constraints. Additionally, the *von Mises yield criterion* may limit the model's ability to capture certain material behaviors, and future work should explore more advanced models like *Deshpande-Fleck's crushable foam* for improved results.

In conclusion, this thesis lays a foundation for further exploration of deep learning models in material simulation. While challenges remain in achieving smooth and reliable training outcomes, the work opens new possibilities for improving the accuracy of battery simulations and extending the approach to other material systems.

6 Outlook

This section outlines six key strategies deemed most relevant and efficient for advancing the research. These methods focus on enhancing the model's generalization, accuracy, and overall performance through the incorporation of experimental diversity, alternative loss function approaches, and optimized training strategies. By addressing current limitations and introducing more advanced techniques, these strategies aim to improve the model's predictive power and robustness for future applications.

1. Incorporate Multiple Experiments for Training

- **Issue:** Using a single test size limits the model's ability to generalize across different scenarios. The physical relationships the model learns may be biased towards specific configurations, reducing its accuracy when applied to varying setups.
- **Aim:** Incorporate data from multiple experiments with different test sizes and configurations. By training the model on diverse experiments, it can better capture the general relationships between strain, stress, and force, improving its ability to generalize to unseen cases.
- **Expected Outcome:** The model will become more robust and versatile, capable of making accurate predictions across different test setups and sizes. This approach will reduce overfitting to a single test size and improve the model's overall performance, especially in scenarios with varying strain or displacement conditions.

2. Investigate Alternative Loss Functions

- **Issue:** The current loss function, such as Mean Squared Error (MSE), may not provide optimal force matching at lower displacements.
- **Aim:** Explore alternative loss functions, such as weighted loss functions or those that consider the Mean Absolute Percentage Error (MAPE), to improve force matching at lower displacements.
- **Expected Outcome:** A more accurate force match across all displacements. Since force error at lower displacements is expected to propagate into higher displacements, weighted loss functions might help mitigate this by focusing on minimizing errors early on. As the number of elements grows with indenter displacement due to the geometry of the test, having more accurate strain predictions in early stages can prevent accumulation of errors at larger displacements.

3. Explore Framework Configuration for Monotonically Converging Training Behavior

- **Issue:** Training configurations often oscillate or diverge after finding a local minimum, instead of adjusting slightly to further optimize it. This can be observed in the training behavior over all epochs, where the model tends to converge to a solution that provides an adequate force match at higher displacements. However, issues arise when the gradients from lower displacements start to influence the model more significantly.
- **Aim:** Identify a framework configuration that ensures monotonically converging training behavior by addressing the imbalance between gradients at lower and higher displacements. The goal is to manage how the gradient for input features (strains) at lower displacements affects model updates without disproportionately impacting the force error.
- **Expected Outcome:** Achieve stable and consistent training progress, reducing the risk of oscillation or divergence. By addressing the local minima issue and controlling how predicted yield stresses for lower strains are adjusted, the model can prevent disproportionate force errors. This is crucial since yield stress at lower strains has a more significant effect on force predictions when fewer elements are involved (lower displacements) and can cause cascading issues as the number of elements grows at higher displacements.

4. Investigate Increasing Element Number / Finer Mesh

- **Issue:** The current element number and mesh size may not provide optimal resolution for the framework's performance.
- **Aim:** Increase the element number for smoother optimization results. Sample gradients when the element count exceeds a threshold to ensure an even gradient distribution across displacements.
- **Expected Outcome:** Achieve smoother edges, filters, and gradient distributions across plastic strains and displacements. Gradient sampling will help mitigate spikes and uneven distributions, though this approach may increase computational cost and slow down simulations.

5. Explore Combination of Selection Methods: Regularization and Enforce Direction

- **Issue:** The regularization selector adds features for 0 strain to ensure a constant number of features per displacement. While this approach leads to fast gradient convergence, it results in reduced accuracy compared to the enforce direction selector, which converges more slowly but provides better accuracy.
- **Aim:** Combine the regularization selector, which maintains constant features per displacement, with the enforce direction method to balance fast convergence and accurate force predictions.
- **Expected Outcome:** Achieve a balance between convergence speed and prediction accuracy. By using both methods, the model should converge faster than using enforce direction alone, while maintaining better accuracy than regularization alone. This ensures a stable number of features per displacement while improving overall performance.

6. Learn Loss Gradient Using a Second Model Instead of an Analytical Solution

- **Issue:** The gradient of the loss with respect to yield stress is typically dependent on the yield surface, making the analytical gradient description limited by the specific yield surface used. Further the analytical solution only gives an approximation of the gradient which might be more complex.
- **Aim:** Use a machine learning (ML) model to describe the gradient, making the loss function independent of the yield surface. The model could learn the relationship between yield stress predictions and resulting force errors by running multiple simulations. All important metrics, such as the complete stress tensor and different equivalent stresses (*vergleichsspannungen*), could be used as inputs to capture more complex relationships.
- **Expected Outcome:** By using a machine learning-based approach, the model could find complex relationships between input features and the resulting gradients that are not captured by the filter mechanism or the analytical gradient description. This double ML approach (using ML for both predictions and gradient calculations) would enable the model to handle more complex optimization scenarios and potentially improve overall accuracy.

Acknowledgements

First and foremost, I would like to express my sincere gratitude to Prof. Dr. Dirk Mohr for granting me the opportunity to conduct my research in his lab. His support has been invaluable in allowing my pursuit of this work.

I would also like to thank my advisors, Thomas Tancogne-Dejean and Xueyang Li, for their insightful discussions, guidance, and encouragement throughout the research process. Their expertise and mentorship have been essential to the development of this thesis.

A special thanks to my co-advisor, Emmanouil Sakaridis, for his technical support and numerous inputs, which greatly contributed to the progress of this work.

I am grateful to the entire team at the *Artificial Intelligence in Mechanics and Manufacturing* lab for providing a collaborative environment that enabled me to carry out this research effectively.

I thoroughly enjoyed my research journey and am particularly grateful for the powerful technology of large language models (LLMs) and AI tools such as DALL-E. These cutting-edge tools enabled me to iterate quickly through my research and rapidly acquire advanced Python skills and machine learning knowledge.

Last but not least, I would like to thank my friends, Malin Karlsson and Max Haberthür, for their invaluable help in proofreading and offering editing suggestions.

References

- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0-387-31073-2.
- Seung Hyun Chung, Thomas Tancogne-Dejean, Juner Zhu, Hailing Luo, and Tomasz Wierzbicki. Failure in lithium-ion batteries under transverse indentation loading. *Journal of Power Sources*, 389:148–159, 6 2018. ISSN 0378-7753. doi: 10.1016/J.JPOWSOUR.2018.04.003.
- PyTorch Community. Setting custom gradient for backward. <https://discuss.pytorch.org/t/setting-custom-gradient-for-backward/111566/5>, 2021. PyTorch Forums, Retrieved from <https://discuss.pytorch.org/t/setting-custom-gradient-for-backward/111566/5>.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Yee Whye Teh and Mike Titterton, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, Chia Laguna Resort, Sardinia, Italy, 2010. PMLR. URL <https://proceedings.mlr.press/v9/glorot10a.html>.
- Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. Neural networks for machine learning, lecture 6a: Overview of mini-batch gradient descent. https://www.cs.toronto.edu/lecture_slides_lec6, 2012. University of Toronto.
- Pejman Khadivi, Ravi Tandon, and Naren Ramakrishnan. Flow of information in feed-forward deep neural networks. *arXiv preprint arXiv:1603.06220*, 2016. URL <https://doi.org/10.48550/arXiv.1603.06220>. arXiv:1603.06220.
- Diederik P Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. URL <https://arxiv.org/abs/1412.6980>.
- PyTorch Documentation. *torch.nn.utils.clip_grad_norm_*, 2023a. Retrieved from https://pytorch.org/docs/stable/generated/torch.nn.utils.clip_grad_norm_.html.
- PyTorch Documentation. *torch.nn.Softplus*, 2023b. <https://pytorch.org/docs/stable/generated/torch.nn.Softplus.html#torch.nn.Softplus>.
- PyTorch Documentation. *torch.nn.Tanh*, 2023c. <https://pytorch.org/docs/stable/generated/torch.nn.Tanh.html>.
- Thomas Tancogne-Dejean and Dirk Mohr. Indentation of small format lithium-ion pouch cells: Experiments and modeling. *International Journal of Solids and Structures*, 257:111900, 12 2022. ISSN 0020-7683. doi: 10.1016/J.IJSOLSTR.2022.111900.
- Jingzhao Zhang, Tianxing He, Suvrit Sra, and Ali Jadbabaie. Why gradient clipping accelerates training: A theoretical justification for adaptivity. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=Bkg6RiCqYQ>.